

Évaluation et Optimisation de mReasoner avec CCOBRA

Ce document retrace la configuration, l'optimisation et l'évaluation du modèle cognitif mReasoner sur le jeu de données humain Ragni2016.csv en utilisant le framework CCOBRA.

1. Fonctionnement de l'Architecture (Code GitHub)

L'implémentation de mReasoner repose sur une architecture hybride conçue pour concilier la fidélité cognitive et la rapidité requise par les benchmarks :

- **Le Cœur Cognitif (LISP)** : Le moteur original de mReasoner est écrit en Common Lisp. Le wrapper Python (mreasoner.py) interagit avec lui en ouvrant un processus enfant (subprocess) Clozure CL et en lui envoyant des commandes textuelles.
- **Le Goulot d'Étranglement** : mReasoner possède 4 paramètres cognitifs ajustables (*epsilon*, *lambda*, *omega*, *sigma*). Interroger LISP en direct pour évaluer chaque variation de paramètre pour chaque humain prendrait des jours.
- **La Solution (Le Cache)** : Le fichier ccobra_mreasoner_cache.py n'appelle pas LISP en direct lors du benchmark. Il charge en mémoire un fichier .npy massif (ex: 2020-09-09-cache-11-10.npy). Cette matrice contient les probabilités de réponses pré-calculées pour les 64 syllogismes, croisées avec **14 641 combinaisons de paramètres possibles**.

2. Création et Configuration du Benchmark

Pour exécuter proprement CCOBRA en ligne de commande, nous avons structuré le projet et résolu les problèmes de chemins de fichiers.

A. Structure des dossiers

Le benchmark a été isolé dans un environnement de travail propre :

Plaintext

```
paul-benchmark/
├── Ragni2016.csv          # Données humaines (139 participants, 64 syllogismes)
├── benchmark.json        # Fichier de configuration CCOBRA
├── models/
│   ├── pymreasoner/      # Le code du modèle adapté
│   │   ├── ccobra_mreasoner_cache.py
│   │   ├── mreasoner/    # Le wrapper LISP original
│   │   └── caches/
│   │       └── 2020-09-09-cache-11-10.npy # Le dictionnaire pré-calculé
```

B. Configuration de CCOBRA (benchmark.json)

Au lieu de passer les arguments manuellement, nous avons créé un fichier de configuration standard :

```
JSON
{
  "type": "adaption",
  "data.test": "Ragni2016.csv",
  "models": ["models/"]
}
```

C. Correction du chemin du Cache

Pour éviter l'erreur FileNotFoundError due aux chemins relatifs de l'invite de commande, nous avons forcé Python à trouver le cache de manière absolue :

```
Python
import os
dir_path = os.path.dirname(os.path.realpath(__file__))
cache_file = os.path.join(dir_path, 'caches', '2020-09-09-cache-11-10.npy')
```

3. Optimisation : Le "Hack" de Vectorisation

Le Problème : Dans le code initial de ccobra_mreasoner_cache.py, la fonction fit() (qui cherche la meilleure "personnalité" pour cloner le participant) utilisait **4 boucles for imbriquées** pour parcourir les 14 641 configurations du cache. En Python pur, cela prenait environ **15 secondes par participant** (soit ~35 minutes pour le fichier complet).

La Solution :

Nous avons remplacé ces boucles par la **vectorisation NumPy**. Au lieu de vérifier les éléments un par un, NumPy multiplie et compare la matrice entière en une seule opération au niveau du processeur.

Extrait de l'optimisation :

```
Python
```

```
# Remplacement des 4 boucles for par des opérations matricielles :
max_preds = self.prediction_cache.max(axis=-1, keepdims=True)
pred_mask = (self.prediction_cache == max_preds)
scores = np.sum(np.mean(train_data * pred_mask, axis=-1), axis=-1)
best_score = scores.max()
best_indices = np.argwhere(np.isclose(scores, best_score))
```

Résultat : Le temps de traitement est passé de 15 secondes par participant à **une fraction de seconde**, réduisant le temps total du benchmark de 35 minutes à quelques secondes seulement.

4. Évaluation Séquentielle et Conclusion

Comment mReasoner est-il évalué ?

CCOBRA utilise une méthode d'évaluation **séquentielle**. Pour chaque participant, mReasoner prédit la réponse à la Question 1, puis CCOBRA lui révèle la vraie réponse (pour qu'il s'adapte), puis il prédit la Question 2, etc. Le modèle tente ainsi de suivre l'évolution psychologique du participant en temps réel.

Interprétation des Résultats :

La commande `ccobra benchmark.json` génère un rapport HTML interactif contenant un *Boxplot* (diagramme en boîte), reproductible et comparable aux graphes scientifiques publiés (ex: [performances.pdf](#)).

- **Médiane obtenue :** ~37,5 % d'accuracy (Précision).
- **Conclusion Cognitive :** Étant donné qu'il y a 9 choix possibles par syllogisme (soit 11,1 % de chance en répondant au hasard), un score de 37,5 % démontre que mReasoner parvient avec succès (et plus de 3 fois mieux que le hasard) à capturer la logique et les biais inhérents au raisonnement humain.